Collaboration, Quality, Test
WS2025/2026
Prof. Dr. Jóakim von Kistowski

Exercise 2.1 – Writing automated Tests

Learning objectives: Understand how to create and run you own automated tests

Today, we write our first automated test in Java. We use the “triangle” example from lecture 2 as the base software to test.

The structure of this exercise is three-fold:

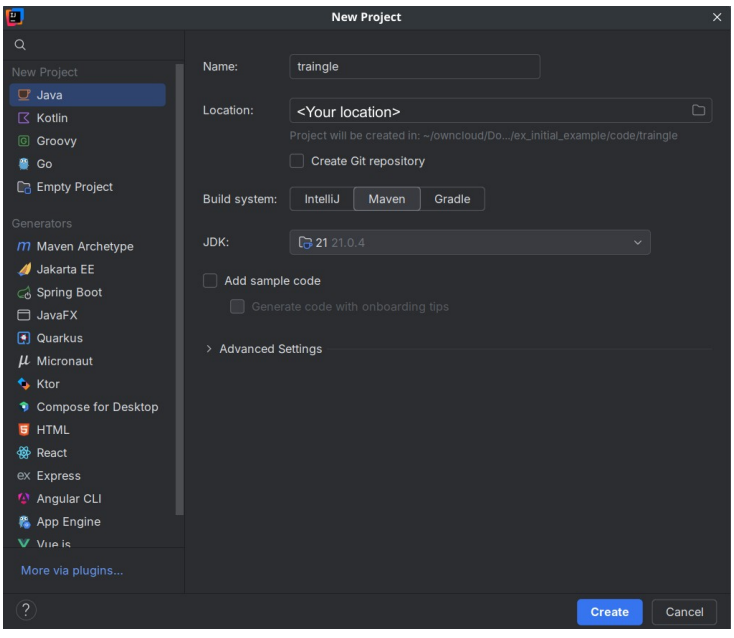
1. Setup of a unit testable development environment
2. Implement the triangle software
3. Implement some unit tests

In this exercise, it is recommended that everyone code on their own device. Feel free to discuss and learn from the experiences of your neighbors.

Part 1 – Setting up Development Project with Unit Tests

Set up a new project using the IntelliJ IDE as follows:

1. (New UI: Click on Menu Button) File → New Project …
2. In the New Project window
    1. Select *Java* in the right hand column
    2. Enter the project’s name “triangle”
    3. Choose a location on your PC
    4. Under *Build system*, select “Maven”
    (we will learn more about that in a later lecture)
    5. For JDK, select the newest JDK on your system (17 or later)
    6. Deselect the *Add sample code* checkbox

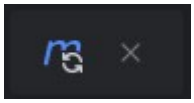


The newly created project should contain several folders, a file called *pom.xml* and a file called *.gitignore* (we can ignore the latter file for now).

7. Open the pom.xml file. Before the file's last line (containing </project>) and enter the following code:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>6.0.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <version>6.0.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

8. Click on the refresh button that appears to the top right or use the keyboard shortcut ctrl+shift+O



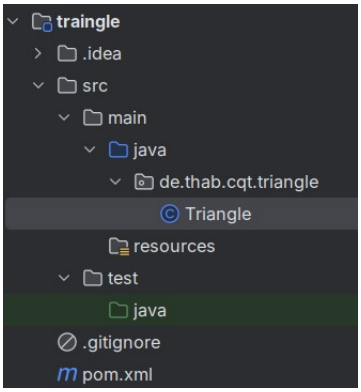
Your project is now set up!

Hint: If IntelliJ gives you trouble importing symbols in your tests, click the m-symbol on the right side of the screen, then click the -button in the newly opened panel.

Part 2 – Implement the Triangle Software

Next, we implement the actual triangle software, keeping in line with some Java development best practices.

1. Create a Java package named *de.thab.cqt.triangle* under the folder *src/main/java*
   Note: Creating packages with a "." in the name contains nested sub-packages.
2. In the newly created package, create a Java class with the name *Triangle*
   Your project should now look like this:



3. Create the following method in your class Triangle:
   `public static int numberOfEqualSides(int lengthA, int lengthB, int lengthC)`
   The method should return the number of sides equal to one another:
   1. For an equilateral triangle: 3
   2. For an isosceles triangle: 2
   3. For an unequal triangle: 1
   4. Invalid inputs should cause the method to return -1

4. Optional, advanced: Instead of returning -1, throw an *IllegalArgumentException*
   Do this if you have finished everything else and still have some time to spare.

Part 3 – Writing Unit Tests

We are going to write a separate unit test for every test case.

1. Create a Java package named *de.thab.cqt.triangle* under the folder *src/test/java*
   Note: Ensure the package name equals the one in *src/main/java*.
2. In the newly created package, create a Java class with the name *TriangleTest*
3. Test that your setup works. Create the following test in the TriangleTest class:

   ```
   @Test
   void testsWork() {
      assertEquals(1, 1);
   }
   ```

   Ensure this test method runs and completes successfully. You can then delete it.
4. Retrieve the list of test-cases you came up with for the triangle example in the lecture (or make a new one).
   Note, for simplicity: You can ignore the test cases where we tested if two sides are as short or shorter than a third side of the triangle.
5. For each test case: Create a unit test and see if it passes.

Exercise 2.2 – Software Quality after ISO 25010

Learning objectives: Foster a deeper understanding of Software Quality Attributes

Task 1 – Example Applications for Software Quality Attributes

Find two example applications for each of the Software Quality attributes listed below. One of the examples should ***not*** be a software application, but a physical object instead (fridge, car, pen, …).

Functional Suitability	Reliability	Usability	Security	Compatibility	Portability	Maintainability	Performance Efficiency

Task 2 – Importance of Software Quality Attributes

2.1.) Describe two reasons as to why it is important to talk about Software Quality Attributes and why they should be prioritized.

_____

_____

_____

_____

2.2.) How could you use ISO 25010 in a practical application.

_____

_____

_____

_____

2.3.) Have a software's good/poor quality ever been exciting/frustrating for you? Name one positive and one negative example?

_____

_____

_____

_____